Guía 09 - Docker y Construcción de Imágenes Seguras en Aplicaciones ASP.NET Core .NET 8

1. **TÍTULO: Guía 09 - Docker y Construcción de Imágenes Seguras en Aplicaciones ASP.NET Core .NET 8**

2. **DATOS INFORMATIVOS**
   - **Asignatura** : Arquitectura Orientada a Servicios
   - **Semestre Académico** : 2026-2
   - **Ciclo** : X
   - **Categoría** : Estudios Electivos de Especialidad (EEE)
   - **Docente** : Mtro. Ing. Luis Alvaro Monzón Condori
   - **Duración** : 04 horas

3. **COMPETENCIAS A SEGUIR.**

   Al finalizar la guía, el estudiante será capaz de:
   - Comprender el concepto de contenedor y su diferencia frente a una instalación tradicional.
   - Explicar el rol de Docker en el desarrollo y despliegue moderno de aplicaciones.
   - Crear imágenes Docker para aplicaciones ASP.NET Core en .NET 8.
   - Elaborar un *Dockerfile* correctamente estructurado.
   - Ejecutar una aplicación web .NET 8 dentro de un contenedor.
   - Verificar el funcionamiento de una API publicada en Docker.
   - Interpretar buenas prácticas básicas para crear imágenes más limpias, portables y eficientes.

4. **REQUISITOS.**
   - Visual Studio 2022 o superior
   - .NET 8 SDK
   - Docker Desktop instalado y operativo
   - Conocimientos de:
     - Programación en C#
     - ASP.NET Core Web API (básico)
     - Línea de comandos básica
     - Estructura de proyectos .NET

5. **BASES TEÓRICAS.**

   5.1. ¿Qué es Docker?

   Docker es una plataforma que permite **empaquetar una aplicación junto con sus dependencias** dentro de una unidad llamada **contenedor**.
   Esto significa que una aplicación puede ejecutarse en distintos equipos manteniendo el mismo comportamiento, reduciendo el clásico problema de: **"En mi máquina sí funciona, pero en otra no".**
   Docker ayuda a que el entorno de desarrollo, pruebas y despliegue sea más uniforme.

   Idea clave:
   Con Docker no solo transportamos el código, sino también parte del entorno necesario para ejecutarlo.
   Ejemplo conceptual:
   Una API ASP.NET Core necesita:

- Runtime de .NET
- Archivos compilados
- Configuración de puertos
- Variables de entorno

Docker permite empaquetar todo eso en una imagen reproducible.

5.2. ¿Qué es un Contenedor?

Un **contenedor** es una instancia en ejecución de una imagen Docker.
Es decir:

- **Imagen:** plantilla o molde
- **Contenedor:** aplicación funcionando a partir de esa plantilla

Analogía simple

- **Imagen Docker** = receta de cocina
- **Contenedor** = plato ya preparado

Puedes crear muchos contenedores a partir de una misma imagen.

5.3. ¿Qué es una Imagen Docker?

Una imagen Docker es un paquete inmutable que contiene:

- El sistema base necesario.
- La aplicación.
- Bibliotecas.
- Configuraciones.
- Instrucciones de ejecución.

La imagen no "se ejecuta sola".
Cuando Docker la inicia, crea un contenedor.

Importancia:
La imagen permite que una aplicación:

- Se distribuya fácilmente.
- Mantenga el mismo entorno.
- Pueda desplegarse en servidores, nube o clústeres.

5.4. Diferencia entre Imagen y Contenedor.

Concepto	Definición	Ejemplo
Imagen	Plantilla construida a partir de un Dockerfile	clinica-api:1.0
Contenedor	Instancia en ejecución de una imagen	API corriendo en puerto 8080

Resumen

- Primero se **construye** la imagen
- Luego se **ejecuta** el contenedor

5.5. ¿Por qué Docker es importante en el desarrollo moderno?

Docker se ha vuelto importante porque permite:

- Uniformidad entre entornos.
- Despliegues más rápidos.

- Menor conflicto por dependencias.
- Portabilidad entre equipos y servidores.
- Mejor preparación para arquitectura basada en microservicios.
- Facilidad para integrarse luego con Docker Compose y Kubernetes.

Caso real
En una institución de salud, una API de citas médicas puede funcionar en la laptop del desarrollador, en el entorno de pruebas y en producción usando la misma imagen base.

5.6. Problema que Docker resuelve.
Sin Docker, una aplicación puede fallar por:

- Versiones distintas de .NET
- Librerías faltantes
- Puertos diferentes
- Variables de entorno mal configuradas
- Diferencias entre Windows y Linux

Docker reduce estos problemas porque estandariza el entorno de ejecución.

Esquema conceptual:

Desarrollo tradicional

- Equipo A → Funciona
- Equipo B → Error de entorno
- Servidor → Comportamiento diferente

Con Docker:

- Misma imagen → Mismo comportamiento esperado en todos los entornos.

5.7. ¿Qué es un Dockerfile?
El *Dockerfile* es un archivo que indica paso a paso cómo construir una imagen.

Contiene instrucciones como:
FROM → imagen base
WORKDIR → directorio de trabajo
COPY → copiar archivos
RUN → ejecutar comandos
EXPOSE → documentar puertos
ENTRYPOINT → comando principal al iniciar el contenedor

Idea importante:
El *Dockerfile* es la "receta" formal para crear la imagen de nuestra aplicación.

5.8. ¿Qué es una Imagen Base?
Una imagen base es el punto de partida de una imagen Docker.

En .NET 8 se suelen usar dos tipos:

- **SDK:** para compilar la aplicación
- **ASP.NET Runtime:** para ejecutar la aplicación ya publicada

Esto da lugar a una práctica muy común llamada: **Multi-stage build**

Consiste en usar varias etapas en el Dockerfile:

- Una etapa para restaurar y compilar.
- Otra etapa más ligera para ejecutar

Beneficios:

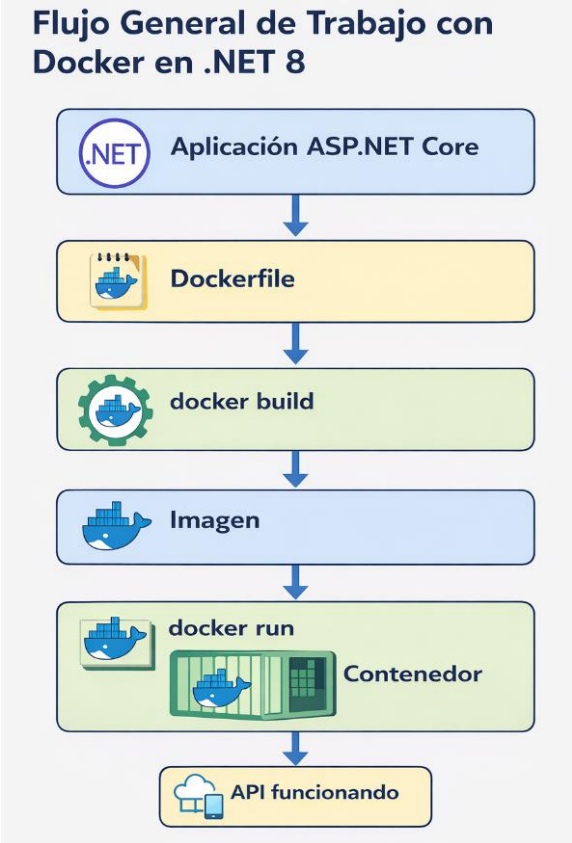
- Imágenes más pequeñas.
- Mejor seguridad.
- Despliegues más rápidos.
- Separación clara entre compilación y ejecución.

Idea clave:
No es recomendable llevar todo el SDK al contenedor final si solo se necesita ejecutar la aplicación.

5.9. Flujo general de trabajo con Docker en .NET 8

El flujo habitual es el siguiente:

- Crear la aplicación ASP.NET Core.
- Crear el Dockerfile.
- Construir la imagen.
- Ejecutar el contenedor.
- Probar la API en navegador, Postman o Swagger.



5.10. Comandos básicos que el estudiante debe conocer:

Comando	Descripción	Ejemplo	Explicación del ejemplo
docker build	Construye una imagen a partir de un Dockerfile	**docker build -t inventario-api:v1 .**	Crea una imagen llamada *inventario-api* con versión *v1* usando el Dockerfile del directorio actual
docker run	Ejecuta un contenedor basado en una imagen	**docker run -d -p 8080:8080 inventario-api:v1**	Ejecuta la imagen en segundo plano y expone el puerto 8080
docker ps	Lista los contenedores en ejecución	**docker ps**	Muestra los contenedores activos
docker ps -a	Lista todos los contenedores (incluidos detenidos)	**docker ps -a**	Permite ver contenedores que ya terminaron
docker images	Lista todas las imágenes disponibles	**docker images**	Muestra las imágenes creadas o descargadas
docker logs	Muestra los logs de un contenedor	**docker logs inventario-api**	Permite ver errores o eventos del contenedor
docker stop	Detiene un contenedor en ejecución	**docker stop inventario-api**	Detiene el contenedor sin eliminarlo
docker start	Inicia un contenedor detenido	**docker start inventario-api**	Vuelve a ejecutar un contenedor detenido
docker rm	Elimina un contenedor	**docker rm inventario-api**	Borra el contenedor (debe estar detenido)
docker rmi	Elimina una imagen	**docker rmi inventario-api:v1**	Elimina la imagen del sistema
docker exec	Ejecuta comandos dentro de un contenedor	**docker exec -it inventario-api bash**	Accede al contenedor en modo interactivo
docker logs -f	Muestra logs en tiempo real	**docker logs -f inventario-api**	Permite monitorear la ejecución en vivo

5.11. Buenas prácticas básicas al crear Imágenes.

- Usar imágenes oficiales.
- Nombrar correctamente las imágenes.
- Mantener el Dockerfile ordenado.
- Preferir compilación en múltiples etapas.
- No incluir archivos innecesarios.
- Probar la aplicación en contenedor antes de desplegar.
- Documentar el puerto y variables relevantes.

6. DESARROLLO DE LA GUÍA:

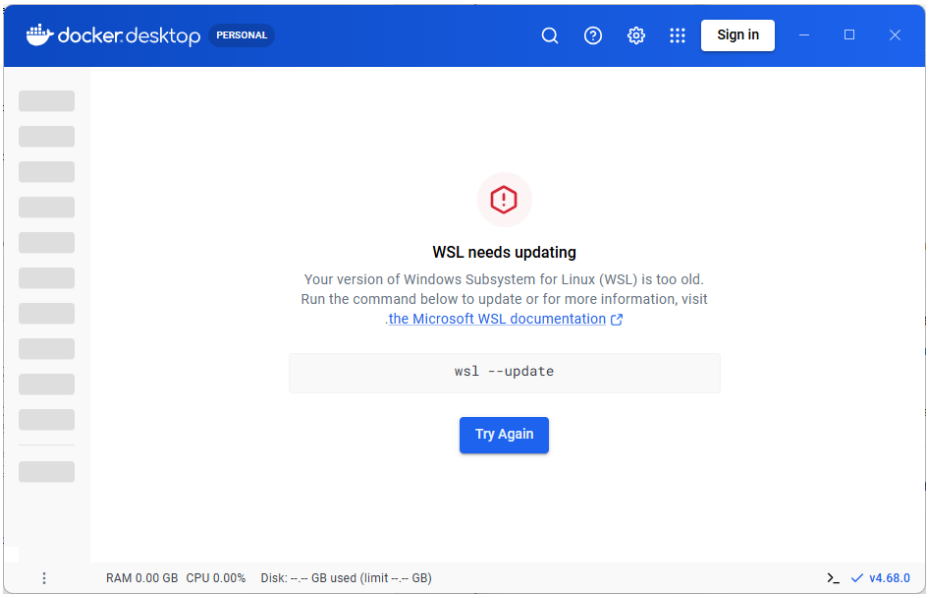
6.1. ACTIVIDAD 00: Instalar Docker

PASO 01: Verificar si Docker está instalado

- Abrir *PowerShell o CMD*
- Ejecutar: *docker --version*
- Si sale el mensaje: *"docker" no se reconoce como un comando interno o externo, programa o archivo por lotes ejecutable."* entonces continuar con el siguiente paso, caso contrario tenemos ya instalado Docker.

PASO 02: Instalar Docker Desktop

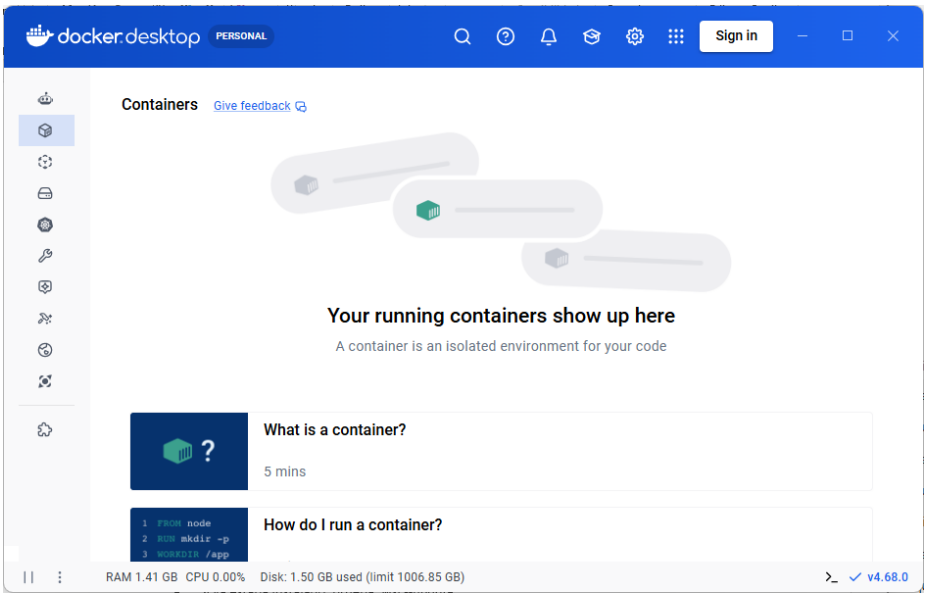
- Descargar desde: https://www.docker.com/products/docker-desktop/
- Durante la instalación: Activar WSL 2 (MUY IMPORTANTE)
- Reiniciar la PC si es necesario.



PASO 03: Si WSL no está instalado o no está activo

- En PowerShell como administrador: *wsl --install*
- Luego reinicia la PC.
- Si ya estaba instalado, prueba: *wsl --update*
- Luego: *wsl --shutdown*
- Después abre otra vez Docker Desktop.

Deberíamos visualizar la siguiente pantalla principal:



6.2. ACTIVIDAD 01: Contenerizar una API ASP.NET Core de Inventario Clínico.

Objetivo:
Crear una API ASP.NET Core .NET 8 para consultar productos médicos y construir su imagen Docker mediante un *Dockerfile*.

Escenario real:
Una clínica desea exponer una API simple para consultar su inventario de insumos médicos:

- Mascarillas
- Guantes
- Alcohol
- Jeringas

La aplicación primero funcionará localmente y luego será empaquetada en Docker.

PASO 01: Crear el proyecto
En Visual Studio:

- Crear un proyecto
- Seleccionar *ASP.NET Core Web API*
- Nombre: *InventarioClinicoAPI*
- Framework: *.NET 8*
- *Habilitar compatibilidad con OpenAPI*

PASO 02: Crear modelo

- Crear carpeta: *Models*
- Crear archivo: *Models/ProductoMedico.cs*

```
namespace InventarioClinicoAPI.Models
{
    // Modelo que representa un producto médico dentro del inventario.
    public class ProductoMedico
    {
        // Identificador único del producto.
        public int Id { get; set; }

        // Nombre del insumo o medicamento.
        public string Nombre { get; set; } = string.Empty;

        // Categoría del producto.
        public string Categoria { get; set; } = string.Empty;

        // Cantidad disponible en almacén.
        public int Stock { get; set; }

        // Precio referencial del producto.
        public decimal Precio { get; set; }
    }
}
```

PASO 03: Crear controlador

- Crear carpeta: *Controllers*
- Crear archivo: *Controllers/InventarioController.cs*

```
using Microsoft.AspNetCore.Mvc;
using InventarioClinicoAPI.Models;
```

```
namespace InventarioClinicoAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class InventarioController: Controller
    {
        // Lista simulada de productos médicos.
        // En esta guía no usamos base de datos todavía.
        // El objetivo es centrarnos en Docker y en la creación de imágenes.
        private static readonly List<ProductoMedico> productos = new()
        {
            new ProductoMedico
            {
                Id = 1,
                Nombre = "Mascarilla KN95",
                Categoria = "Protección",
                Stock = 250,
                Precio = 3.50m
            },
            new ProductoMedico
            {
                Id = 2,
                Nombre = "Guantes de látex",
                Categoria = "Protección",
                Stock = 500,
                Precio = 1.20m
            },
            new ProductoMedico
            {
                Id = 3,
                Nombre = "Alcohol medicinal",
                Categoria = "Desinfección",
                Stock = 180,
                Precio = 7.00m
            },
            new ProductoMedico
            {
                Id = 4,
                Nombre = "Jeringa 10 ml",
                Categoria = "Procedimiento",
                Stock = 320,
                Precio = 0.80m
            }
        };

        // Devuelve la lista completa del inventario.
        [HttpGet]
        public ActionResult<IEnumerable<ProductoMedico>> Get()
        {
            return Ok(productos);
        }

        // Devuelve un producto específico según su Id.
        [HttpGet("{id}")]
        public ActionResult<ProductoMedico> GetById(int id)
        {
            var producto = productos.FirstOrDefault(p => p.Id == id);

            if (producto == null)
            {
                return NotFound(new
                {
                    mensaje = "Producto no encontrado."
                });
            }

            return Ok(producto);
```

```
        }
    }
}
```

PASO 04: Ejecutar el proyecto

- Ejecutar el proyecto
- Verificar el correcto funcionamiento.

PASO 05: Crear archivo Dockerfile

- Crear archivo en la raíz del proyecto: *Dockerfile*

```
# ----------------------------
# ETAPA 1: BUILD
# ----------------------------
# Usamos la imagen SDK de .NET 8 para restaurar, compilar y publicar.
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build

# Definimos el directorio de trabajo dentro del contenedor.
WORKDIR /src

# Copiamos primero el archivo del proyecto para aprovechar la caché de Docker.
COPY ["InventarioClinicoAPI.csproj", "./"]

# Restauramos las dependencias NuGet del proyecto.
RUN dotnet restore "InventarioClinicoAPI.csproj"

# Copiamos el resto de archivos del proyecto.
COPY . .

# Publicamos la aplicación en modo Release en una carpeta de salida.
RUN dotnet publish "InventarioClinicoAPI.csproj" -c Release -o /app/publish
/p:UseAppHost=false

# ----------------------------
# ETAPA 2: RUNTIME
# ----------------------------
# Usamos una imagen más ligera solo para ejecutar la aplicación.
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final

# Directorio donde vivirá la aplicación publicada.
WORKDIR /app

# Copiamos desde la etapa build únicamente los archivos ya publicados.
COPY --from=build /app/publish .

# Indicamos el puerto que usará la aplicación dentro del contenedor.
EXPOSE 8080

# Variable de entorno para que Kestrel escuche en el puerto 8080.
ENV ASPNETCORE_URLS=http://+:8080

# Comando de inicio del contenedor.
ENTRYPOINT ["dotnet", "InventarioClinicoAPI.dll"]
```

Explicación del Dockerfile

Parte 1: build

Compila la aplicación usando el SDK.

Parte 2: final

Prepara un contenedor más ligero que solo ejecuta la API.

¿Por qué se hace así?

Porque el contenedor final no necesita herramientas de compilación, solo el runtime.

PASO 06: Construir la imagen
Abrir terminal en la raíz del proyecto y ejecutar:

```
docker build -t inventario-clinico-api:v1 .
```

Explicación:

- docker build → Construye la imagen
- -t → Asigna nombre y etiqueta
- inventario-clinico-api:v1 → Nombre de la imagen
- . → Usa el Dockerfile del directorio actual

PASO 07: Verificar imagen creada
En el mismo terminar ejecutar:

```
docker images
```

PASO 08: Ejecutar el contenedor
En el mismo terminar ejecutar:

```
docker run -d -p 8080:8080 --name inventario-api-contenedor inventario-clinico-api:v1
```

Explicación

- -d → Modo segundo plano
- -p 8080:8080 → Mapea puertos
- --name → Asigna nombre al contenedor

PASO 09: Verificar ejecución
En el mismo terminar ejecutar:

```
docker ps
```

Luego probar: http://localhost:8080/api/inventario

PASO 10: Revisar logs

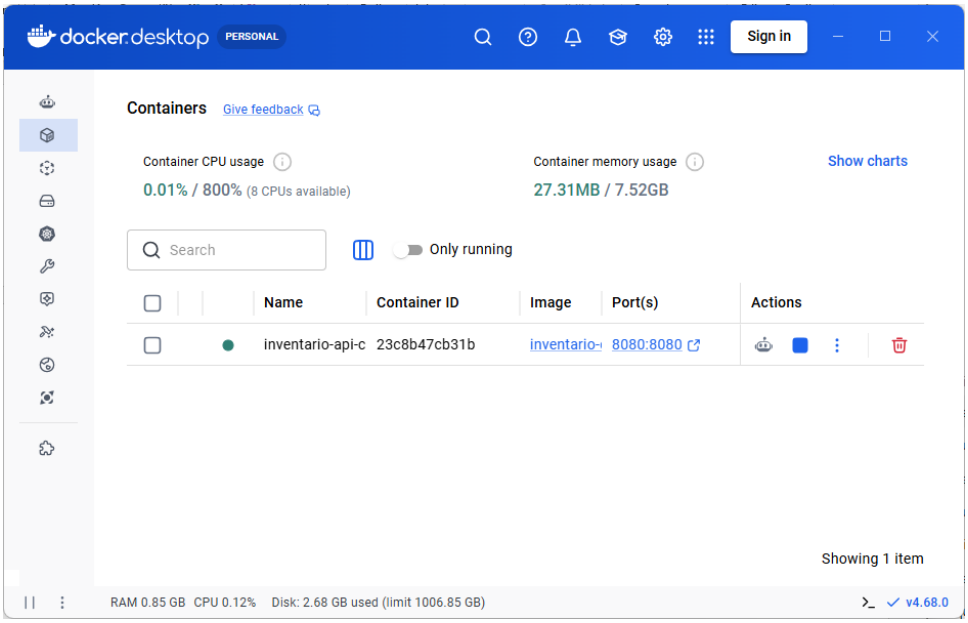
```
docker logs inventario-api-contenedor
```

Esto ayuda a identificar errores de arranque o de configuración.

Resultado esperado

El estudiante habrá logrado:

- Crear una API funcional en .NET 8.
- Construir una imagen Docker.
- Ejecutar la API dentro de un contenedor.
- Consumir la API desde el navegador.



Visualización del contenedor generado

6.3. ACTIVIDAD 02: Contenerizar una API ASP.NET Core de Gestión de Pedidos de Farmacia.

Objetivo:
Desarrollar una API ASP.NET Core .NET 8 para registrar pedidos de farmacia y crear su imagen Docker.

Escenario real:
Una farmacia clínica quiere un servicio simple para:

- Listar pedidos
- Registrar nuevos pedidos
- Consultar pedidos por id

La meta es contenerizar una API con operaciones más cercanas a un caso real de negocio.

PASO 01: Crear proyecto
En Visual Studio:
ASP.NET Core Web API
Nombre: *FarmaciaPedidosAPI*
Framework: .NET 8
Activar Swagger

PASO 02: Crear modelo
Crear carpeta: *Models*
Crear archivo: *Models/PedidoFarmacia.cs*

```
namespace FarmaciaPedidosAPI.Models
{
    // Modelo que representa un pedido realizado al área de farmacia.
    public class PedidoFarmacia
    {
        // Identificador del pedido.
        public int Id { get; set; }
```

```
        // Nombre del medicamento solicitado.
        public string Medicamento { get; set; } = string.Empty;

        // Área que solicita el medicamento.
        public string AreaSolicitante { get; set; } = string.Empty;

        // Cantidad requerida.
        public int Cantidad { get; set; }

        // Fecha del pedido.
        public DateTime Fecha { get; set; }

        // Estado actual del pedido.
        public string Estado { get; set; } = string.Empty;
    }
}
```

PASO 03: Crear controlador

Crear archivo: *Controllers/PedidosController.cs*

```
using FarmaciaPedidosAPI.Models;
using Microsoft.AspNetCore.Mvc;

namespace FarmaciaPedidosApi.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class PedidosController : ControllerBase
    {
        // Lista en memoria que simula pedidos de farmacia.
        private static readonly List<PedidoFarmacia> pedidos = new()
        {
            new PedidoFarmacia
            {
                Id = 1,
                Medicamento = "Paracetamol 500 mg",
                AreaSolicitante = "Emergencia",
                Cantidad = 100,
                Fecha = DateTime.Now.AddDays(-1),
                Estado = "Pendiente"
            },
            new PedidoFarmacia
            {
                Id = 2,
                Medicamento = "Suero fisiológico",
                AreaSolicitante = "Hospitalización",
                Cantidad = 80,
                Fecha = DateTime.Now,
                Estado = "Atendido"
            }
        };

        // Lista todos los pedidos.
        [HttpGet]
        public ActionResult<IEnumerable<PedidoFarmacia>> Get()
        {
            return Ok(pedidos);
        }

        // Busca un pedido por Id.
        [HttpGet("{id}")]
        public ActionResult<PedidoFarmacia> GetById(int id)
        {
            var pedido = pedidos.FirstOrDefault(p => p.Id == id);
```

```
            if (pedido == null)
            {
                return NotFound(new
                {
                    mensaje = "Pedido no encontrado."
                });
            }

            return Ok(pedido);
        }

        // Registra un nuevo pedido.
        [HttpPost]
        public ActionResult<PedidoFarmacia> Post([FromBody] PedidoFarmacia
nuevoPedido)
        {
            // Validación básica de negocio.
            if (string.IsNullOrWhiteSpace(nuevoPedido.Medicamento))
            {
                return BadRequest(new
                {
                    mensaje = "El nombre del medicamento es obligatorio."
                });
            }

            if (nuevoPedido.Cantidad <= 0)
            {
                return BadRequest(new
                {
                    mensaje = "La cantidad debe ser mayor que cero."
                });
            }

            // Generación simple de Id.
            nuevoPedido.Id = pedidos.Max(p => p.Id) + 1;

            // Si no se envía fecha, se asigna la actual.
            if (nuevoPedido.Fecha == default)
            {
                nuevoPedido.Fecha = DateTime.Now;
            }

            // Si no se envía estado, se define por defecto.
            if (string.IsNullOrWhiteSpace(nuevoPedido.Estado))
            {
                nuevoPedido.Estado = "Pendiente";
            }

            pedidos.Add(nuevoPedido);

            return CreatedAtAction(nameof(GetById), new { id = nuevoPedido.Id },
nuevoPedido);
        }
    }
}
```

PASO 04: Verificar Program.cs

Ejecutar el proyecto y verificar el correcto funcionamiento de la aplicación.

PASO 05: Crear Dockerfile

Crear en la raíz del proyecto el archivo *Dockerfile*

```
# Etapa de compilación
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
```

```
# Copia inicial del archivo de proyecto para restaurar dependencias
COPY ["FarmaciaPedidosAPI.csproj", "./"]
RUN dotnet restore "FarmaciaPedidosAPI.csproj"

# Copia del resto del proyecto
COPY . .

# Publicación en modo Release
RUN dotnet publish "FarmaciaPedidosAPI.csproj" -c Release -o /app/publish
/p:UseAppHost=false

# Etapa final de ejecución
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final
WORKDIR /app

# Copia de los archivos publicados
COPY --from=build /app/publish .

# Puerto expuesto por la aplicación
EXPOSE 8080

# Configuración del servidor web interno
ENV ASPNETCORE_URLS=http://+:8080

# Punto de entrada
ENTRYPOINT ["dotnet", "FarmaciaPedidosAPI.dll"]
```

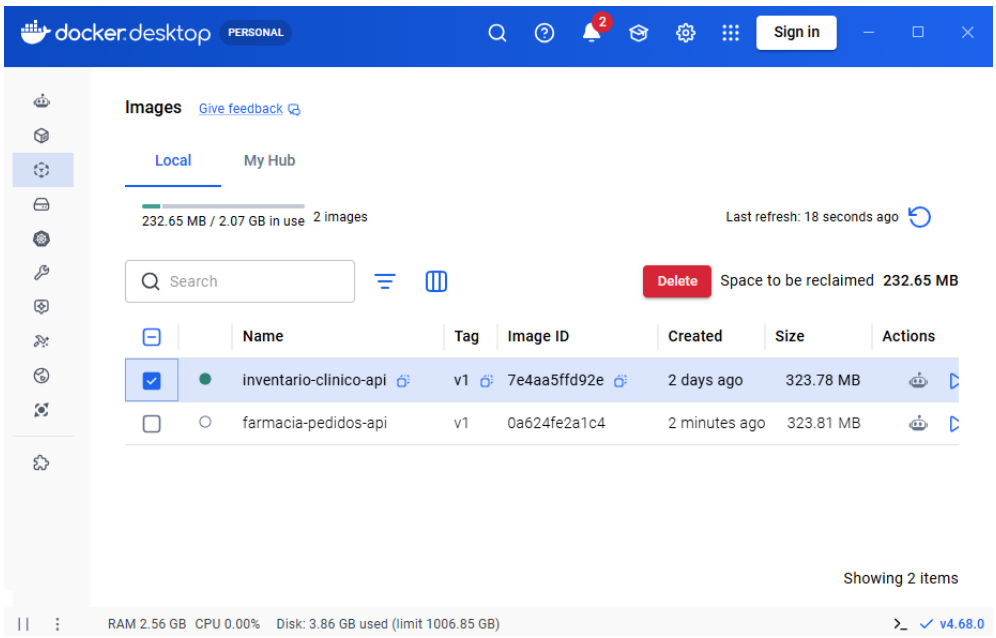
PASO 06: Construir imagen

Inicializar Docker Desktop

Ingresar el comando en el Terminal del proyecto:

```
docker build -t farmacia-pedidos-api:v1 .
```

Verificar la creación de la imagen:

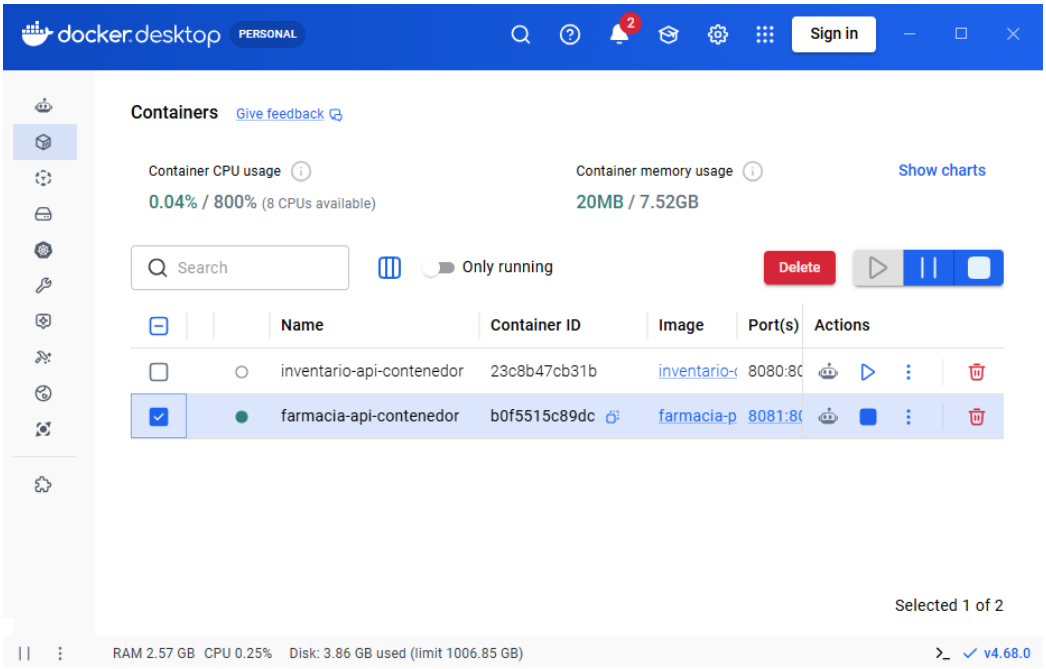


PASO 08: Ejecutar contenedor

Ingresar el comando en el Terminal del proyecto:

```
docker run -d -p 8081:8080 --name farmacia-api-contenedor farmacia-pedidos-api:v1
```

Verificar la creación del Contenedor:



PASO 09: Verificar ejecución
En el mismo terminar ejecutar:

```
docker ps
```

Luego probar: http://localhost:8081/api/pedidos

PASO 10: Revisar logs

```
docker logs farmacia-api-contenedor
```

Esto ayuda a identificar errores de arranque o de configuración.

PASO 10: Comprobar funcionamiento real
Registrar un pedido desde Swagger y luego listar nuevamente los pedidos.

Observación:
Si al ejecutar el proyecto nos muestra error al querer verificar la sintaxis del archivo Docker entonces, tenemos que deshabilitarlo para que no sea considerado al momento de ejecutar nuestra aplicación, realizando los siguientes pasos:

- Clic derecho sobre el archivo *Dockerfile*.
- Seleccionar *Propiedades*
- Cambiar la propiedad Acción de compilación a *Ninguno.*

7. EJERCICIOS PROPUESTOS:

7.1. EJERCICIO PROPUESTO 01: Contenerización + lógica de negocio real.

Enunciado:

- Desarrollar una API ASP.NET Core .NET 8 que permita registrar pacientes en un sistema de triaje y determinar automáticamente su nivel de prioridad médica.
- Posteriormente, contenerizar la aplicación utilizando Docker y verificar su funcionamiento.

Especificaciones funcionales:

Entidad: *PacienteTriaje*

Debe incluir:

- Id
- Nombre
- Edad
- Temperatura
- FrecuenciaCardiaca
- NivelPrioridad (se calcula automáticamente)

Reglas de negocio:

El sistema debe determinar el nivel de prioridad:

Crítico

Temperatura > 39

FrecuenciaCardiaca > 120

Urgente

Temperatura entre 37 y 39

FrecuenciaCardiaca entre 90 y 120

Normal

Caso contrario

Endpoints requeridos:

GET /api/triaje

POST /api/triaje

Requisitos Docker:

- Crear Dockerfile correctamente estructurado
- Construir imagen
- Ejecutar contenedor
- Probar endpoints desde Swagger del contenedor

Validación obligatoria:

El estudiante debe demostrar:

- Registro de paciente
- Cálculo automático de prioridad
- Consulta desde el contenedor

7.2. EJERCICIO PROPUESTO 02: API de Control de Stock Crítico (Alerta Médica).

Enunciado:

- Desarrollar una API que permita registrar productos médicos y detectar automáticamente cuáles se encuentran en estado crítico.
- La API deberá ser ejecutada dentro de un contenedor Docker y analizar el comportamiento de los datos en memoria.

Entidad: *ProductoCritico*
Campos:

- Id
- Nombre
- Stock
- StockMinimo
- Estado (calculado)

Reglas de negocio:
El estado se calcula:

Crítico
Stock < StockMinimo
Alerta
Stock = StockMinimo
Normal
Stock > StockMinimo

Endpoints:
GET /api/stock
POST /api/stock
GET /api/stock/criticos

Requisitos Docker:

- Contenerizar la aplicación
- Ejecutar en puerto diferente (ej: 8082)
- Validar endpoints desde el contenedor

8. PROCESO DE EVALUACIÓN:

Criterios de evaluación de la guía:

Criterios de evaluación	Descripción	Puntaje por Ítem
1. Actividad 01	Desarrollo de la Actividad 01 de la Guía	05
2. Actividad 02	Desarrollo de la Actividad 02 de la Guía	05
3. Ejercicio Propuesto 01	Desarrollo del Ejercicio Propuesto 01	05
4. Ejercicio Propuesto 02	Desarrollo del Ejercicio Propuesto 02	05
TOTAL		**20**